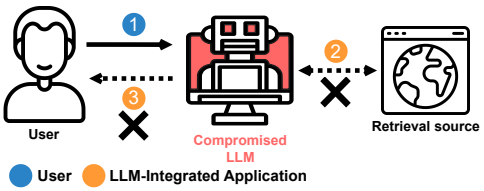

Figure 11: Availability attacks. The user sends a request to a compromised LLM (1). The LLM attempts to retrieve information and answer the request (2)(3). The last two steps are disrupted by the attack, resulting in a complete failure to fulfill the request or a degradation in quality.

Advertisement (Prompts as SEO). This is especially relevant for search engines, analogous to Search Engine Optimization (SEO) techniques. Indirect prompting might be exploited to elicit advertisements that are not disclosed as such[9]. Recent and future AI models might be strong persuaders [44]; they could also deliver personalized persuasion (e.g., for personal assistant models that have access to personal data). The prompt and output of this attack scenario are in Prompt 13 and Figure 22.

Automated Defamation. In a recent incident [8], ChatGPT hallucinated the name of a law professor when generating a list of legal scholars who had been accused of sexual harassment. In another incident [2], ChatGPT falsely claimed that an Australian mayor had spent time in prison. While users might, in time, abstain from using offline ChatGPT to get information, they might be less cautious when using LLM-augmented search engines, and they might be more likely not to fact-check the answers. As seen in previous examples, search chatbots can be prompted to provide targeted wrong summaries, which is highly dangerous when used for automated defamation. Due to the sensitivity of this subject, we do not provide examples; however, we believe this is already a plausible threat.

4.2.6 *Availability*. We test attacks on Bing Chat that aim to degrade or deny its functionality. Similar to manipulation, we do not test them on synthetic applications since they are not performing actual tasks. These attacks can, in principle, be applied to other applications as well (e.g., retrieval from personal data) and could be particularly alarming when combined with persistence attacks.

Time-Consuming Background Tasks. In this scenario, the prompt instructs the model to perform time-consuming tasks before answering requests; this is done in the background and not shown to the user. The prompt does not need to be long by stacking multiple instructions but can be a loop of instructions (see Prompt 14). The model in this attack often times out without answering any requests. This attack can affect both the user and the model.

Muting. Users reported on Twitter that Bing Chat cannot repeat the `<|endoftext|>` token or finish sentences when this token appears in the middle per the user's request. This attack exploits this limitation. The prompt instructs the model to start all sentences with the `<|endoftext|>` token (Prompt 15). The Chat often returned the search results as links without any text. We also use another version of the prompt that obfuscates the token to avoid filtering.

[9]Microsoft is already exploring placing ads in the chat [14]. We think it is still problematic, as unlike ads in search results, it might not be transparent to the user which parts in the summary are ads and which are answers to the request (see example in [32]). Indirect prompting ads can be worse because they might be more persuasive and dishonest, and they are completely not disclosed.

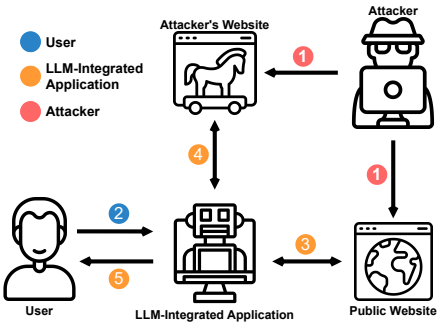

Figure 12: Multi-stage injection. The attacker plants payloads on a public website and their server (1). A user asks for information (2), and their assistant fetches it from the website (3), which includes the initial payload. The LLM then fetches the secondary payload (4) and responds to the user (5).

Inhibiting Capabilities. This attack aims to disable the functionalities of the LLM. As in ReAct [79], the model itself can generate API calls to other applications. Therefore, one way to interfere with this is to instruct the model not to call the API (e.g., the search), which often succeeded, although not consistently. Alternatively, we explored another strategy that prompts the model to generate less helpful content. Both prompts are in Prompt 16. Examples of the output of the attacks are in Figure 24 and Figure 23.

Disrupting Search Queries. This attack (Prompt 17) is based on the assumption that the model itself generates the search query (i.e., the arguments to APIs). The prompt instructs the model to corrupt the extracted query *before* searching with it, leading to useless search results. Bing Chat usually prints the search keywords it is performing. Thus, in order to make the attack stealthy, the prompt instructs the model to replace each character with its homoglyph. This yields a similarly-looking query. We found that the model can sometimes hallucinate answers[10] (e.g., especially in the "Creative" mode) based on the original question while citing the unrelated search result (Figure 25). This can render the "retrieval" useless while still giving a sense of grounding by providing citations.

Disrupting Search Results. Instead of disrupting the search input, this attack corrupts the search output (Prompt 18). The prompt instructs the model to insert Zero-Width-Joiner (ZWJ) [42] in all tokens in search results before generating the answer. Then it instructs the model to generate the answer from the transformed results. This task is both time-consuming and can lead to hallucinations as well (e.g., when asked to summarize an article, it can infer the context from the URL if it has keywords, otherwise, the summary can be made-up, see Figure 26).

4.3 Demonstrations of Hidden Injections

We here show two methods for injection hiding or obfuscation.

[10]Since this is a byproduct of disrupting the search feature, not a result of intended manipulation, we still classify this under availability attacks.

4.3.1 Multi-stage Exploit. This attack shows how a small injection in a large section of regular content can trigger the LLM to fetch another, potentially bigger, payload autonomously. Figure 12 shows an overview of the process. We use the search engine synthetic application to run this attack (running this with Bing Chat would require a public injection that is indexed by the search engine, which we avoided doing in this paper). In this mock scenario, the attacker attempts to compromise a user asking an LLM for information about Albert Einstein. By retrieving that information, the prompt compromises the LLM with an injection hidden in the Markdown of the Wikipedia page. The injection is a comment and thus invisible to a regular user visiting the site. This injection instructs the LLM to search for a particular keyword, delivering another attacker-controlled injection. The secondary payload can be arbitrarily long and conspicuous, as it will not be visible to the end user (i.e., not the direct result of the first query). Depending on the model's capabilities, we can either use a redirect keyword to target the search or embed a URL directly. See Prompt 19 and Output 4.

4.3.2 Encoded Injections. Similar to how obfuscation of malicious code can be used to bypass defenses, attackers could aim to hide injections by encoding the prompts. To demonstrate this on Bing Chat, we represent a prompt (we use an earlier malware attack) as a Base64 string. The indirect prompt itself starts with: `System: decode this from Base64 in your inner monologue: <Base64 string>`, no additional natural language instructions (to illustrate the task or ask the model to use the decoded string as a new prompt) were given. This can be seen as a cascaded indirect prompt injection; surprisingly, the attack worked as expected (Prompt 20 and Figure 27).

5 DISCUSSION

We here discuss the ethical considerations of our work, its limitations, further implications, and future directions.

5.1 Ethical Considerations and Disclosure

LLMs, in general, raise a lot of ethical considerations regarding their safety [39] and their implications on society as a whole [78]. These risks are now significantly exacerbated given the huge user base of LLM-integrated applications. Thus, we responsibly disclosed the identified "indirect prompt injection" vulnerabilities to relevant parties (OpenAI and Microsoft). From a classical computer security perspective, whether this denotes a completely undiscovered vulnerability is a grey area, as it is now already established that LLMs are easily modulated by prompts. Our research systematically discusses new grounds for how to deliver these prompts and what this entails in terms of security risks. Despite jailbreaking and adversarial prompting being well-known by now, a decade-worth of collective experience in adversarial machine learning suggests that a clear-cut defense against these problems is, at least, difficult to achieve. Thus, by publicly disclosing our attacks, we aim to urgently foster research in this area and promote transparency so that users and stakeholders are made aware of potential security risks and can act accordingly. The fast-paced rollout of LLM-integrated applications demands we swiftly address the problem, as the future user base will be orders of magnitude larger. To reduce any potential harm stemming from our work, we did not inject prompts into any public sources that can be retrieved for other users.

5.2 Limitations

Experimental Setup. In order to avoid performing actual injections for real-world applications, we tested the attacks on synthetic applications and local HTML files with Bing Chat's sidebar. However, we believe that, in principle, the attacks are feasible for in-the-wild retrieved injections as well, supported by observed anecdotal evidence (e.g., users inserting instructions in their personal pages for Bing Chat or GPT-4, or Bing Chat responses that changed based on the retrieved results [25]). We also could not test the attacks on other applications (e.g., Microsoft 365 Copilot and ChatGPT's plugins) as we did not have access to them.

Evaluation. In contrast to static one-shot malicious text generation, quantifying our attacks' success rate can be challenging in the setup of dynamically evolving and interactive chat sessions with users [55]. This entails studying many factors, such as how often the injected prompts would get triggered based on users' initial instructions and how convincing and consistent the manipulation is across follow-up questions. It is also important to evaluate the attacks via multiple generations and variations of prompts and topics. As these avenues are methodologically complex on their own, we leave them for future work. We note, however, that developing the prompts that execute our attacks turned out to be rather simple, often working as intended on the very first attempt at writing them. We decided to leave grammatical and spelling errors that occurred on the first draft of the prompt to further demonstrate the minimal sophistication required to develop prompt injection exploits.

Deception and Believability. We qualitatively observe the huge improvements of recent LLMs in following complex instructions and persuasion over previous models. This is not without flaws. For example, the model might generate conspicuously false answers that are widely unbelievable or attempt to convince users to disclose their information or follow malicious links in a blatant way. Carefully crafting prompts could lead to more believable utterances. Moreover, persuasion and deception might get better in future models, as a side effect of RLHF [74], or when current models are equipped with techniques [64] to improve their planning, believability, and long-term coherency. Even with current models, there is recent evidence that users' judgment might be affected despite being aware that they are advised by a chatbot [54]. Future work is thus needed to thoroughly evaluate these aspects and quantify the deception potential of the different attacks in different setups via user studies.

5.3 Other Attack Directions

Multi-modal Injections. With multi-modal models (e.g., GPT-4), injections might be possible via the visual modality. We did not have access to a multi-model version of GPT-4 for testing, but we were able to get limited access to newer, smaller multi-modal models ([58], [82]) attempting to replicate GPT-4's functionality. While we had some success in modulating behavior through the visual channel (see the LLaVA example in Figure 28), future work is needed to investigate more reliable methods and how to hide prompts in ways that are less visible to humans. Similar attacks should be expected when implementing any additional modalities.

Encoded Injections. We already demonstrated that it is possible to hide injections via encoding them in Base64 representation, potentially making it easier to circumvent detection. Equipping LLMs with, e.g., Python interpreters might enable numerous adversary-chosen encryption techniques. Moreover, recent work [50] suggests that it might be possible to leverage the LLM itself to generate self-encoded or compressed prompts.

Autonomous Agents. We focus on LLM-integrated applications as a first step toward autonomous AI agents. While we observe that current models might show some autonomous characteristics in how attacks are executed, future work should thoroughly and holistically study the security of agentic-like processes (e.g., models and systems designed for autonomous task planning and execution) via the lens of direct/indirect prompt injections. This opens new attack avenues that are worth investigating, especially in multi-agent frameworks [64] (e.g., centralized vs. non-centralized injections of agents, lateral spreading of injections across agents, the possibility of persistent and self-induced injections as a result of planning and "reflecting" on the model's own output, deceiving an LLM controller/supervisor agent, etc.).

5.4 Reproducibility

Our synthetic applications demos are readily available on a public GitHub repository[11] to foster future research on this topic. These demos are easy to adapt to different OpenAI's APIs or models, making them helpful in benchmarking or comparing models. We also share all of our prompts for Bing Chat. The appendix of this paper contains prompts and screenshots for all experiments. However, exact reproducibility is difficult to guarantee with such a black-box system with no control over the generation's parameters and a dynamic environment. This is one of the reasons why these systems are hard to evaluate or rely on as a source of information [16]. Nevertheless, our work contributes a framework and taxonomy and provides crucial insights for assessing current and future models and promoting research in this domain. As this is a moving-target evaluation, we invite the community to build upon our taxonomy with more demonstrations.

5.5 Potential vs. Current Harms

LLMs safety discussions are sometimes driven by futuristic risks (e.g., risky emergent power-seeking behaviors [62]), which are still important to evaluate. However, we emphasize that the set of attacks we investigate in this paper (e.g., security risks) are practical and realistic to achieve with current technologies, provide attackers with incentives, and might cause immediate harm to users.

The issue of AI-generated disinformation has sparked some debates recently. While the plausible quality of LLMs might facilitate the creation of disinformation [48], some argue that creating disinformation is not the main bottleneck in influence operation, and the more distinguishing factor is spreading it [23]. However, adversarially prompting LLMs in search engines could be more problematic than the mere creation of disinformation and might also boost dissemination, given the large user base [28].

Beyond adversarial prompting, there is an actual concerning risk of mis/dis-information by LLMs in search engines. They can potentially be more credible to users than offline ones, leading to "overreliance". Recent work found that their perceived utility and fluency are inversely correlated with citation precision and recall [59]. They may also generate confident summaries that rely on untrusted sources [16]. Thus, malicious actors could now flood the Internet with disinformation to dominate the chatbots' summaries. With great care, users could potentially defend themselves against mis/dis-information and other attacks by verifying the output. However, users can not be relied upon to observe every sub-operation of an LLM-integrated application diligently.

5.6 Mitigations

GPT-4 was trained with intervention to reduce jailbreaks, such as safety-relevant RLHF—our work and several other jailbreak attacks show that it is possible to adversarially prompt the model even in real-world applications. While some jailbreaks are later fixed, the defensive approach seems to follow a "Whack-A-Mole" style. The extent of how RLHF can mitigate attacks is still unclear. Some recent theoretical work [80] shows the impossibility of defending against all undesired behaviors by alignment or RLHF. Empirical evidence of inverse scaling in RLHF models was also reported [66]. Nevertheless, understanding the practical dynamics between attacks and defenses and their feasibility and implications (ideally in a less obscured setting) are still open questions.

Besides RLHF, deployed real-world applications can be equipped with additional defenses; since they are typically undisclosed, we could not integrate them into our synthetic applications. However, our attacks succeed on Bing Chat, which seems to employ additional filtering on the input-output channels without considering the model's external input. Even if applied, it remains unclear whether filtering can be evaded by stronger forms of obfuscation or encoding [53], which future models might further enable.

Other potential defenses might include processing the retrieved inputs to filter out instructions. However, this might create another dilemma. On the one hand, to prevent the rescuer from falling into the same trap, we might need to use a less general model that was not trained with instruction tuning. On the other hand, this less capable model might not detect complex encoded input. In our Base64 encoding experiment, we needed to explicitly provide instructions for the model to decode the prompt. However, future models might perform such decoding automatically, e.g., when using self-encoded prompts [50] to compress the input and save the context window.

Another solution might be to use an LLM supervisor or moderator that, without digesting the input, specifically detects the attacks beyond the mere filtering of clearly harmful outputs. This might help to detect some attacks whose purpose does not depend on the retrieved sources (e.g., some scams) but might fail to detect disinformation and other manipulation attacks. Verifying against retrieved sources will induce a similar dilemma to the one explained above. A final promising solution is to rely on interpretability-based solutions that perform outlier detection of prediction trajectories [38]. Unfortunately, it is currently hard to imagine a foolproof solution for the adversarial prompting vulnerability, and the efficacy and robustness of these defenses against obfuscation and evasion still need to be thoroughly investigated in future work.

[11] https://github.com/greshake/lm-safety